ChatGPT

Plan for Building the Qwirkle Connect Score Calculator

Step 1: Understand the Game Rules and Scoring

Begin by familiarizing yourself with **Qwirkle Connect** rules and how scoring works. Qwirkle is a tile-laying game where players create lines of tiles that are all the same **shape** or all the same **color**, with no duplicates in a line [1]. A line can have at most 6 tiles (since there are 6 shapes and 6 colors total). If you complete a line of 6 unique tiles, that's called a **Qwirkle**, which earns a bonus.

- **Initial board setup:** Qwirkle Connect uses a 16×16 grid board divided into 4 interchangeable quadrants [2] [3]. At the start, 24 tiles are randomly placed face-up on **black anchor squares** (6 per quadrant) [4] [5]. These starting tiles serve as anchors to build from, and each black square always starts occupied. Some board squares are marked with **bonus values** (+1 or +2) – each quadrant has ten +1 and two +2 squares in fixed relative positions [6]. (The quadrants may be rotated or swapped, so part of your task is to determine their orientation from the black anchor tile pattern.)

- **Placement rules:** On each turn, a player places one or more new tiles in a single straight row or column. All newly placed tiles must form a continuous line with no gaps, and share one attribute (all same color or all same shape) without duplicates [1]. They also must connect to the existing tiles on the board (at least one side touching a previously placed tile) [7]. These rules ensure every move is a valid line extension or new line. (For our project we assume all moves given are valid, so you won't have to check rule compliance in code, but it's good to know for context.)

- **Scoring rules:** When a player places tiles, they score points for every line that is created or extended by that move [8]. In particular:

  - You get points equal to the **number of tiles in a line** you form or extend. For example, adding to form a line of 4 tiles scores 4 points for that line [9].
  - If your move completes a line of all 6 shapes or 6 colors (a **Qwirkle**), you earn a **+6 bonus** points [10]. (A full line of 6 yields 6 points for the tiles + 6 bonus, totaling 12 [11].)
  - If you place multiple tiles, you may score for multiple lines: the primary line of your tiles, plus any perpendicular lines formed at each new tile. Each such line counts separately [12].
  - **Bonus squares:** If a new tile is placed on a bonus cell (marked "1" or "2" on the board), add that bonus to the score [13]. For a +1 cell, add 1 extra point; for +2, add 2 points. **Importantly**, a bonus for a given tile is counted only once, no matter how many lines that tile is part of [14]. (In other words, if a tile lies on a bonus square and contributes to two lines, you still only add the +1 or +2 once for that tile.)

To illustrate, consider a simple scoring example: a player places a single tile – a green circle – such that it connects into two lines: one line of green tiles and one line of circle-shaped tiles. Suppose each of those lines has 2 tiles (including the new one). The player scores 2 points for the green line + 2 points for the circle

line, and if the green circle was placed on a “+1” bonus square, +1 extra point, totaling **5 points** [15]. If instead a move completes a six-tile line, the player scores 6 for the line plus the 6-point Qwirkle bonus (12 points total) [11].

Understanding these rules is critical, as our system will need to identify lines of tiles and calculate scores accordingly.

Step 2: Set Up the Data Pipeline and Explore the Dataset

Next, get comfortable with the provided dataset and directory structure. The materials include four directories: **antrenare** (training), **testare** (testing), **evaluare** (evaluation), and **imagini auxiliare** (auxiliary images) [16]. For now, focus on the training data:

- **Training images:** In `antrenare/` you have images capturing the board state after each turn for 5 sample games. Each game `j` has images named `j00.jpg, j01.jpg, ..., j20.jpg`. `j00.jpg` is the initial setup after the 24 anchor tiles are placed, and moves 1–20 are `j01.jpg` through `j20.jpg` [17]. In total there are 105 training images (5 games × 21 images each) [18].

- **Training annotations:** For each move image (except the initial `j00`), there’s a corresponding `*.txt` annotation file with the same base name (e.g. `101.jpg` → `101.txt`). The annotation for image `jXX.jpg` lists the new tile(s) placed in that move and the score earned [19] [20]. (No annotation file exists for `j00` since no move was made yet [19].) We’ll discuss the exact format in Step 8, but it contains the coordinates, shape, color of each new tile, and the move’s score.

- **Auxiliary images:** The `imagini auxiliare/` folder contains helpful reference images, including a photo of the empty board (no tiles) and images with **all possible tiles placed on the board** [21]. Use these to understand board layout and to extract visual data for shapes and colors. For example, one auxiliary image shows the board with every shape/color combination tile laid out – you can use this to sample each shape’s appearance and each color.

Loading and inspecting data: Start by writing a small script to load an image using OpenCV and display it or print basic info:

```
import cv2
img = cv2.imread('antrenare/100.jpg')
print(img.shape)
cv2.imshow('Board', img); cv2.waitKey(0)
```

This will confirm you can access the images. You should see that the camera is fixed (the board appears roughly in the same place each time) [22]. It’s a good idea to visualize a couple of images (including the empty board) to get a sense of the board boundaries, cell grid, and tile appearance.

Also, read one of the annotation text files to see how information is recorded. For example, open `101.txt` in a text editor. You will see entries like `E5 6B 12` (just an illustrative format) – we’ll decode this later.

Verifying the annotation contents against the image will help you understand what the system needs to output.

Step 3: Board Grid Detection and Calibration

Goal: Map the pixel coordinates of the image to the 16×16 board grid (rows 1–16, columns A–P). This will allow you to translate image locations into board positions (like "5D") and vice versa.

Steps to achieve this: - **Find the board in the image:** Since the camera is fixed and the board likely fills most of the frame, you might hardcode the region of interest or detect the board's edges. If the board has a distinct border, you can use edge detection (Canny) or color segmentation to find it. Another approach is to use the auxiliary empty board image as a template: - Convert the empty board image to grayscale and apply a slight blur. - Use Hough line detection to find straight grid lines. You should detect a set of evenly spaced vertical and horizontal lines (15 interior grid lines each way, dividing the 16 columns and 16 rows). - If Hough lines are too sensitive to noise, you can instead locate the four corner points of the board (perhaps by looking for large changes in color or using the fact that the board likely contrasts with the table/background). Once you have corners, you can apply a perspective transform to get a top-down aligned view. However, because the camera was top-down fixed, a transform might not be necessary – the image is probably already nearly aligned.

- **Determine cell coordinates:** Using either the detected grid lines or known board dimensions, compute the pixel coordinates corresponding to each cell center or corners. For example, if the top-left corner of the board is at pixel `(x0, y0)` and bottom-right at `(x1, y1)`, then the cell width ≈ `(x1 - x0) / 16` and height ≈ `(y1 - y0) / 16`. You can then map a grid position like row 5, col D to pixel region:
- `px = x0 + (col_index * cell_width)`
- `py = y0 + (row_index * cell_height)` (Use integer indices where A→0, B→1, … P→15 for columns, and 1→0, …, 16→15 for rows.)

- **Identify special squares (anchors and bonuses):** We know each quadrant has identical special square layouts [3]. The six black anchor squares in each quadrant are filled in the initial image by the 24 starting tiles. You can use `j00.jpg` to locate them:

- Detect all tiles in the initial image (since no moves yet, all tiles present are the 24 anchors). Perhaps threshold for dark areas – the anchor tiles might appear as mostly black (with colored symbols).
- Record their positions (should be 24 distinct grid cells). These correspond to the black square locations. By comparing these coordinates to the expected pattern of anchor positions in a quadrant, you can deduce how the quadrants are arranged and rotated.

For example, if a quadrant's black squares should form a particular pattern (you might know from rules or the auxiliary full-board image how the black squares are arranged in one quadrant), you can match that pattern against the detected anchor coords in each 8×8 section of the board. This tells you which quadrant piece is where and its rotation. Once you know that, you can programmatically know which squares are +1 or +2 in the overall board: - The assignment notes there are 10 (+1) and 2 (+2) bonus squares per quadrant [6]. If you know a certain cell in a quadrant is, say, the +2 in that quadrant's design, you can translate it to the board's coordinates. - It might be easier to hardcode the relative locations of bonuses in a quadrant (for

instance, perhaps the +2 squares are at certain row/col offsets in each quadrant). Then using the quadrant orientation, compute the actual board cells that are +1/+2. Store these in a set or dictionary for quick lookup during scoring.

- **Verify grid mapping:** As a sanity check, overlay the grid on an image. You can draw lines on the image at the computed row/column boundaries or mark cell centers. Use a few known reference points (like the corners or anchor tiles) to ensure your mapping is correct. The mapping must be accurate so that when you detect a tile in pixel space, you can confidently convert it to the correct row/column label.

By the end of this step, you should have functions or data structures to convert between pixel coordinates and board coordinates. For example, a function `find_cell(x, y)` that returns the board cell (row, col) for a given pixel point. You should also have identified the special cells (anchors, +1, +2), possibly via quadrant analysis.

Step 4: Detecting New Tiles in Each Move

Now comes the core vision task: given two consecutive board states (before and after a move), identify which tile(s) were added.

Approach 1: Image differencing. Since tiles are only added (never moved or removed) between images, a straightforward method is to compare the current move's image with the previous image: - Load the previous image (board state before the move) and the current image (after the move). - Convert them to grayscale (and maybe apply a slight Gaussian blur to reduce noise). - Compute the absolute difference: `diff = cv2.absdiff(prev_gray, curr_gray)`. - Threshold the diff to get a binary mask of "changes." This will highlight regions where new tiles have appeared (which should show up as bright differences). - You may need to filter out minor differences due to lighting changes. For example, use a somewhat high threshold to ignore small pixel value changes, or perform morphological opening on the diff mask to remove tiny speckles. - The result should be blobs corresponding to the new tile(s). Find connected components or contours in this diff mask. Each significant contour likely corresponds to a newly placed tile.

Approach 2: Direct detection of tiles in current image and subtraction of known ones. You can maintain a representation of the board state as you iterate through moves. For example, keep a 16×16 boolean grid of which cells are occupied after each move. At the start (after initial placement) you know 24 cells are occupied. For each new move image, you could attempt to detect **all** tiles on the board, then subtract the ones you already knew about, leaving the new ones: - To detect all tiles on a single image, you could look for the black square shape of tiles. Qwirkle tiles are black with a colored symbol. Against the board (which is lighter), a tile might appear as a roughly square dark region with a colorful center. You could threshold for low brightness to catch black tile regions. - Alternatively, threshold for the colored shapes (which are high-saturation areas) and then dilate a bit to encompass the tile. - This might be more complex than the diff approach, but it's another viable strategy especially if image differencing is noisy.

In practice, image differencing is very effective here because the camera is fixed [22] – new tiles will "pop out" in the diff. Just be mindful of any global illumination shifts (if one image is slightly brighter overall, many pixels change a little). If that's an issue, you can try a normalized difference or ignore low-intensity changes: - One trick: apply adaptive histogram equalization or color normalization on both images before

diff, to account for brightness variation. - Or convert images to HSV and only take difference of the value channel, etc., depending on what proves robust.

Once you have a mask of new tile regions, determine each new tile's **grid position**: - Compute the centroid of each contour or the bounding box. Then use your grid mapping from Step 3 to convert that centroid (pixel x,y) to a row/column coordinate. - Alternatively, find which grid cell each contour's area falls into. If your grid cell coordinates are known, you can check a small region around each cell center in the diff mask to see if a new tile is present. - It's wise to double-check: the number of new tiles should match the number of contours found. In Qwirkle, a player can place 1–6 tiles in a move (6 at most, if they complete a Qwirkle). Most often it will be 1 or a few. If your detection finds an unexpected number (like 0 or too many), you may need to adjust thresholds or handle noise.

Example: If `diff` reveals one strong blob, that indicates a single tile was placed. If it reveals, say, three distinct blobs in a line, that indicates three tiles placed contiguously. They should lie in one row or column (you can verify this after determining coordinates as a sanity check per placement rules).

After this step, you will have the **set of new tile positions** for the move (e.g. "row 7 col D" and "row 7 col E" for two tiles, or just "row 5 col B" for one tile, etc.). Keep track of these positions and also update your internal board state to mark them as occupied going forward.

Tip: If using the difference method, be sure to handle the first move (j01 vs j00) carefully. The `j00` initial image has those 24 tiles; `j01` will have those plus whatever was added in move 1. The diff of j01 minus j00 should ideally highlight only the move 1 tiles. If initial lighting or board alignment differences cause the anchors to appear in the diff, you may need to mask out the anchor positions (since you know them from initial state) when analyzing the diff.

Step 5: Classifying Tile Shapes and Colors

For each newly detected tile, the next task is to identify **what shape and what color** it is. Qwirkle has 6 shapes and 6 colors. Our system must output each new tile as a shape number 1–6 and color letter (as specified by the assignment) [23].

Shapes in Qwirkle: 1 = circle, 2 = clover (four-leaf clover), 3 = diamond, 4 = square, 5 = four-point star, 6 = eight-point star [23].

Colors in Qwirkle Connect: R = red, B = blue, G = green, Y = yellow, O = orange, W = white [23]. *(Note: In the classic game, there was purple instead of white, but our dataset uses white.*)

Extracting the tile's symbol: Since the tile itself is black and the symbol is colored, a good strategy is to isolate the colored shape: - Crop the image to the region of the new tile (for example, take a bounding box around the contour from the diff or a fixed 1-cell-sized crop using the grid coordinates). - Convert that region to HSV color space for easier segmentation. In HSV, the black tile background has very low value (V) and the colored shape has higher value. - Create a mask for the shape by filtering out low-value (dark) pixels. For instance, keep pixels where V is above a threshold or where saturation is high. This should remove most of the black background and leave the colored shape region. You can also specifically exclude very dark pixels (the tile edges) by a threshold on V. - At this point, you might perform morphological

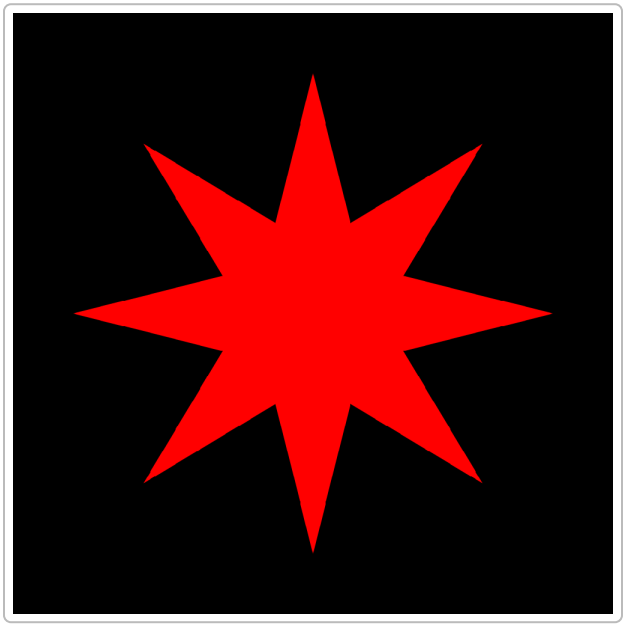
operations on the mask: e.g. a closing to fill any small holes (sometimes the printing on tiles might have slight gaps or glare). - Use `cv2.findContours` on this mask to get the outline of the shape.

Now, classify the contour shape: - One approach is **template matching**: Using the auxiliary "all pieces" image, you can collect an example of each shape. For example, find a red circle, red clover, etc., or any color since shape is what matters (you could binarize those references as well). Then use `cv2.matchShapes` or compare Hu moments of the contours【71†】. OpenCV's `matchShapes` gives a similarity measure between two contours – you can pick the shape class with the lowest difference. - Another approach: look at simple features of the contour: - **Circle** will have a single smooth round outline (you could check contour perimeter vs area – a circle has the maximum area for a given perimeter). - **Square vs Diamond:** Geometrically, a diamond is just a rotated square. If tiles are not rotated by players, a square's contour will be axis-aligned (vertical/horizontal edges) while a diamond's will be at 45°. But since players might place tiles at random orientations, distinguishing these two might be tricky. If you notice in images that tiles are mostly not rotated (symbols might be upright), you can partially rely on that. Otherwise, the square and diamond contours will both have four corners – you might differentiate by angle or moment features. - **Stars:** The 4-point star (looks like an "X") has four pointed arms and 4-fold symmetry; the 8-point star has eight points. These will have more complex contours with multiple sharp angles. You could count the number of convex peaks in the contour or use Fourier descriptors. However, a robust way again is template matching – the 8-point star shape will not match well with the 4-point star template. - **Clover:** The clover shape is like a club (♣) with 4 lobes. Its contour might appear as four connected circles, which is fairly distinct (it has a **curvy outline with three bulges**). It will have no sharp angles (unlike stars), but more lobes than a circle.

Using `matchShapes` is convenient: For each known shape contour (from a template), compute the match score with the new tile's contour and choose the smallest score. Before doing this, you might normalize scale (the tiles should all be same size though) and maybe rotate templates if needed (but Hu moments used in `matchShapes` are rotation invariant by default). Ensure the contour you feed in is one whole shape (if your mask had noise, you might accidentally get multiple contours – in such a case, take the largest contour which should be the shape).

Classifying color: Once you have the shape, determining color is easier: - Compute the average color of the tile's symbol region. You can use the mask from earlier to extract pixels of the symbol and average their BGR or HSV values. - In HSV, check the hue. The six colors likely cluster around specific hue values: - Red: hue ≈ 0 (or 180) – careful since red wraps around the hue axis. - Orange: hue ≈ 15–30. - Yellow: hue ≈ 30–60. - Green: hue ≈ 60–90 (depending on shade). - Blue: hue ≈ 100–140. - White: saturation will be very low, value high (hue can be arbitrary since it's nearly neutral). - An alternative is to prepare reference color ranges. For example, from the auxiliary image with all tiles, measure the HSV of one tile of each color. Then during detection, assign the color whose reference is closest in HSV space (or within a threshold range). - White can be detected by a simple condition: if saturation < some low threshold and value is high, classify as W. The other colors will have saturation high. Purple is not used here (if it were, it'd have hue ~140-160, but in our case purple is replaced by white). - Note: Make sure to distinguish red and orange – under some lighting, a dark orange could appear "red". Using hue is usually sufficient (red will be around 0 or 180°, orange around 20°), but if needed you could also consider the tile's brightness (red vs orange might differ slightly). Given consistent lighting and the distinct printed colors, this shouldn't be a big problem.

Now you can label each new tile with its shape number and color letter. For example, you might determine a new tile is a red eight-point star – that corresponds to shape 6 and color R, so the code is `6R`. Another might be a blue circle = `1B`, and so on.



Example: After segmentation, the red eight-point star shape is isolated from its black tile. The system would recognize this as shape "6" (eight-point star) [23] . The hue of the symbol indicates the color "R" (red). So this tile's identity would be **6R**.

Finally, update your internal board representation with the identified shape/color at that cell (if you are keeping such a representation). This could be useful for cross-checking moves or for visualizing the state.

Step 6: (Optional) Verify Move Validity as a Sanity Check

Although the problem guarantees all moves are valid, it's good practice to use the game rules to double-check your detections: - **Uniformity:** All new tiles you found in this move should share the same shape or the same color. If you detect two new tiles and one is a red circle (1R) and the other a blue square (4B), something's wrong – that combination wouldn't form a valid line. This could mean a mis-detection (perhaps the second "tile" was actually part of something else). - **Connectivity:** The new tiles should all lie in one row or one column (forming a continuous line). If your new tile coordinates are not aligned or have a gap between them, that's a red flag. - **No duplicates:** If multiple tiles in the move have the same shape and same color, that's invalid (e.g. two identical red circles in one move can't happen). Ensure the shapes and colors of the new tiles follow the no-duplicate rule for that line.

Implementing these checks can help catch errors. For instance, if due to noise your diff algorithm falsely thought there were tiles at two positions that are not actually both new, the rules might be violated and you'd know to adjust parameters. In general, if a move fails a rule check, you should reconsider the image processing result for that move.

Since in a valid game these conditions will always hold, your output should inherently satisfy them when correct. Thus, any violation indicates an issue in detection/classification that you can investigate.

Step 7: Calculating the Score for the Move

With the new tiles identified and classified, computing the score is the final step. We will mimic how a human scores a Qwirkle move, following the official rules: 1. **Identify all lines formed or extended by the new tiles.** There are two kinds of lines to check: - The **primary line**: all the new tiles themselves will form a straight line (by rule, they share a row or column). Include any existing tiles at the ends of this line as well, because if your new tiles attach to an existing line, that line is now longer. Count the total number of tiles in this full line (existing + new). - **Secondary lines**: for each new tile, look in the perpendicular direction (up/down if the main line is horizontal, or left/right if main line is vertical). That tile might extend or create a line in the perpendicular direction by connecting with previously existing tiles above or below it (or left/right of it). If so, count the number of tiles in that perpendicular line. - Essentially, treat each new tile as a potential cross-point forming one additional line. In Scrabble terms, these are like "cross words" formed by the newly placed letters.

1. **Score each line by its length.** For each line found in step 1 that has length ≥ 2:
2. Add points equal to the number of tiles in that line [9]. (If a new tile didn't connect to any tile in the perpendicular direction, that line length is 1, and you score nothing for a line of length 1.)
3. If the line length is 6, that's a Qwirkle – add **+6 bonus** points [10] on top of the 6 points. Each Qwirkle line (if you happened to complete two lines of 6 in one move, that would be two separate 6-point bonuses).
4. **Add any bonus square points.** For each new tile that landed on a bonus cell, add the appropriate bonus [24] [14]. If a tile is on a "1" cell, add +1; on a "2" cell, add +2. As noted before, even if that tile contributes to two lines, you do *not* double-count the cell; it's one bonus per tile [14]. In practice, if you handle bonuses per tile (not per line), you'll naturally do this correctly.
5. **Sum up** all the points from lines and bonuses to get the total move score.

Let's go through a quick example. Suppose after analyzing the move, you found two new tiles were placed at E5 and E6 (just an example): - They lie in the same column E, so the primary line is column E from row 5 to 6 (plus any existing tiles contiguous in col E). Say there were two existing tiles at E3 and E4 above them. That makes the vertical line E3-E4-E5-E6 = 4 tiles. This line scores 4 points. - Now check secondary lines for each new tile: - Tile at E5: look at its row (row 5). Imagine it forms a horizontal line with an existing tile at D5. Then the line D5-E5 has 2 tiles, score 2 points. - Tile at E6: look at row 6. If no adjacent tiles, then no secondary line (or line length 1, score 0). - No Qwirkle was formed (lines length were 4 and 2, not 6). - Suppose E5 was on a +1 bonus cell (just as an example). Then add 1. - Total score = 4 + 2 + 0 + 1 = **7 points**.

This matches the rule: each new line formed (4-length and 2-length) scored their length, and the bonus was added. (You can cross-check this with provided examples or ground truth annotations to ensure the logic matches exactly how the instructor expects scoring.)

Make sure to use your known set of bonus cell coordinates to check for bonuses.

One more example from the problem description: A player places three tiles in one move and creates three lines – one of 4 tiles and two of 2 tiles – and two of those tiles land on +1 cells. The score would be *4 (for the 4-tile line) + 2 + 2 (for the two 2-tile lines) + 1 + 1 (for the two bonuses) = 10 points*. This aligns with the kind of examples given [25] [26].

By implementing the scoring carefully, you ensure your output not only identifies the tiles but also provides the correct score, which will be validated by the evaluation script.

Step 8: Generating the Output Annotation File

For each move image, you will output a `.txt` file describing that move's added tiles and score, in the exact format required. According to the specification, the annotation file contains, in order: - The **positions** of the newly added tiles, sorted left-to-right, top-to-bottom (i.e. sort by row, then by column) [27]. - The **shape and color** of each new tile, using the code numbers/letters described earlier (shape 1–6, color R/B/G/Y/O/W) [23]. - The **score** achieved by that move [27].

All this information is given in a single line (for each move) in the text file. In other words, for each new tile you output `<Position> <ShapeColor>`, and finally the score.

Format details: - **Position:** The board uses rows numbered 1–16 (top to bottom) and columns labeled A–P (left to right). The position is written as `<row><column>`. For example, row 5 column D becomes `5D`. Row 12 column A becomes `12A`. (Single-digit rows don't get a leading zero, so 5D is correct, not 05D.) - If multiple tiles were placed, list each position in ascending order: first by row (smallest number first, which is higher up on the board), and if the row is the same, by column (A comes before B, etc.) [27]. - **Shape and color:** Immediately after each position, give the shape number and color letter. For example, `5D 6R` would mean at 5D there is a tile with shape 6 and color R (red eight-point star). Do not separate shape and color with a space – it should be a single token like `6R`. - If multiple tiles, you'll have multiple `<Position> <ShapeColor>` pairs in one line. Typically they are separated by a space. - After listing all new tiles, the last entry in the line is the **score** for that move as an integer.

Example: Let's say in image `203.jpg` (game 2, move 3) two tiles were placed: a red circle at B7 and a blue square at C7, and the move scored 4 points. The annotation line should be:

```
7B 1R 7C 4B 4
```

Breaking that down: `7B 1R` (at 7B a shape-1 red), `7C 4B` (at 7C a shape-4 blue), and `4` (score 4). The positions are listed B before C since B is to the left of C.

Check your output format against the training annotations to be sure. The assignment document even showed an example annotation for image 101.jpg in Figure 6 [28]. Once you produce some output, compare it with the ground truth `.txt` of that move to see if your spacing and ordering match exactly. Small format deviations (like extra spaces or wrong ordering) could cause the evaluator to mark it incorrect.

Some additional tips for output: - Make sure you **do not include the initial state** (image `j00`) in your outputs. You only generate `j01.txt` through `j20.txt` for each game. There is no `j00.txt` (the

evaluation script expects none for those) [19] . - Each annotation is one line of text. Don't add extra lines or comments. - Use uppercase letters for column and color (as specified). - Double-check that shape numbers are 1-6 (as digits) and not words or other encoding.

By structuring the output correctly, you ensure your solution's files will be read properly by the evaluation program.

Step 9: Testing and Evaluation Procedure

With the core functionality implemented, test your system thoroughly on the training data before running on the test set: - **Run on training games:** For each image in `antrenare/`, generate the annotation and compare it to the provided ground truth `.txt` file. You can write a small script to automate this comparison for all files. - Use the provided evaluation script `evalueaza_solutie.py` (likely in the `evaluare/` directory) to verify your solution's accuracy. This script is designed to take your output files and compute an accuracy score against the ground truth annotations [29] . Typically, you might use it like:

```
python evalueaza_solutie.py --predictions=path_to_your_output --ground-
truth=antrenare/
```

(Check any readme or comments in the evaluator code for exact usage.) The script will read each of your `.txt` files and compare to the expected file for each image, reporting any mismatches. It will likely check that positions, shape/color, and score all match exactly, down to the character, awarding points for each correct tile identification and score [30] . - Perform a "fake test" by holding out some training images (or using the auxiliary images in a creative way) and see if your system can generalize. For example, shuffle one of the game's moves and see if the logic still picks up new tiles correctly without relying on sequential consistency (the difference method inherently uses sequential images, so that's fine). - **Validate edge cases:** Look at moves with the maximum number of tiles (did any move place 6 tiles? – likely if someone got a Qwirkle). Ensure your sorting and output include all 6 and the score includes the Qwirkle bonus. Also test a move that plays only 1 tile (there are many); does your code handle the single-contour case properly including secondary line scoring? - The evaluation script may also give a breakdown of scores: e.g. 0.01 points for each correct score and some points for correct tile identification [30] . Use that to guide improvements. If your tile positions are right but a score is off, focus on the scoring function. If a score is right but a shape/color code is wrong, debug the classification.

Remember that the test set (`testare/`) has the same format as training but with different games. Your code should not use any ground truth from training except for development. Once you're confident on training data, run it on the test images to produce `*.txt` outputs for each, which you will submit. (The actual test images might not be provided until the submission phase as noted in the assignment, so ensure your code is robust.)

During evaluation, consistency is key: even a small format issue or one wrong character will likely be marked incorrect. That's why replicating the exact output format and thoroughly testing is so important.

Step 10: Common Challenges and Practical Tips

Finally, be aware of some common issues and utilize OpenCV techniques to handle them:

- **Lighting variations:** As mentioned, images might have slight brightness differences [31]. If you find the diff approach flagging many false changes, consider normalizing brightness. Converting to HSV and using only the hue or saturation for detecting colored shapes can mitigate brightness issues in classification. For diff, focusing on Value channel changes may help (since a new black tile greatly lowers the value in that region).

- **Tile rotations:** If players rotate the tiles when placing, the shape orientation in the image can vary. A square tile placed diagonally will appear as a diamond shape, potentially confusing your shape classifier. To counter this, use rotation-invariant methods (Hu moments via `matchShapes` are rotation-invariant for example). If you rely on corner counting, note that a square and a diamond both have 4 corners; you'd need another property to tell them apart. In practice, you might observe that most tiles are placed upright (you can confirm by looking at the symbols in the images). But it's good to handle arbitrary rotation if possible.

- **Shape ambiguity:** The four-point star vs clover could be confused if segmentation is poor (both have four "arms" if clover's lobes are interpreted wrong). Ensure your mask is clean. Use morphological operations (erode/dilate) to separate adjacent shapes if, say, tiles are touching (though tiles in Qwirkle are usually placed with a small gap, so that likely isn't an issue).

- **Contour detection issues:** Sometimes the colored shape might have internal holes (e.g. the clover shape might have a small gap in the center). `findContours` might then find multiple contours (outer outline and inner hole). You can set the retrieval mode to `RETR_EXTERNAL` to get only external borders if you've filled the shape properly. Or simply take the contour with largest area assuming that's the outer boundary.

- **Color thresholds:** Calibrate your HSV thresholds for color carefully using the auxiliary images. For instance, red tiles might appear in the image with hue near 0 or 179 (since hue wraps around). You may need to account for that wrap-around (e.g., treat hue < 10 or >170 as red). White tiles will have very low saturation, so you might classify by `if sat < 30 and val > 200: color = W`. Testing on known pixels from training images will help fine-tune these ranges.

- **Performance:** The image processing here is not heavy – a few 600x600-ish images and contour analyses. It should run quickly. Just be mindful to not do extremely expensive operations for every tile (but even if you do, with ~100 images it's fine). If you use template matching via brute force correlation, that can be slow; better use contour matching or a one-time computation of templates.

- **Documentation:** Keep track of how you derived special positions (like how you mapped quadrant bonus squares). It's easy to mix up coordinates, so double-check using a known example (you could simulate placing a tile on a known +2 square in a training image and verifying your code would indeed add 2).

- **Evaluation pitfalls:** When you run `evalueaza_solutie.py`, if it reports mismatches, pay attention to what they are. If it says a position is wrong, maybe your sorting was off or numbering. If it says shape/color wrong, check if you possibly mis-numbered shapes (e.g. mistaking the numbering order). Use the provided mapping from the PDF [23] as the authoritative reference for shape IDs and color letters. For instance, the eight-point star is 6 (not 8), which is a detail to get right.

By addressing these issues and iterating, you'll build confidence in the system. Each part of the pipeline (detection → classification → scoring → formatting) can be developed and tested independently to some extent: - You can test shape/color classification on the auxiliary "all tiles" image first (iterate through known positions in that image, classify, and see if you get the expected 36 combinations). - You can test the difference detection on a pair of consecutive training images to see if it correctly isolates the new tile. - You can test the scoring on a known scenario (even manually create a small board state in code and simulate a move).

Throughout development, leverage OpenCV visualization functions to debug (draw bounding boxes, put text on images for predicted labels, etc.). This will help you intuitively see if the system's intermediate results make sense.

By following this plan – understanding the problem, implementing stepwise from detection to output, and rigorously testing – you'll gradually build a robust solution for automatically scoring Qwirkle Connect moves. Good luck, and have fun applying computer vision to this board game challenge!

Sources:

- Official Qwirkle Connect rules and scoring examples [15] [14]
- Assignment PDF (CAVA 2025 Tema 1) – data description, format specs [27] [23]
- UltraBoardGames – Qwirkle Connect overview (for understanding bonus squares and initial setup) [13] [12]

---

[1] [2] [3] [4] [6] [7] [8] [9] [10] [16] [17] [18] [19] [20] [21] [22] [23] [25] [26] [27] [28] [29] [30] [31] Tema1 (1).pdf
file://file_000000006510724699bb9c01b09e00df

[5] [12] [13] [24] How to play Qwirkle Connect | Official Rules | UltraBoardGames
https://www.ultraboardgames.com/qwirkle/qwirkle-connect.php

[11] Qwirkle - Wikipedia
https://en.wikipedia.org/wiki/Qwirkle

[14] [15] Qwirkle_EXp_Rules_130312
https://tesera.ru/images/items/1321993/Qwirkle_Expansion_1.pdf